



Common Patterns

Backtracking 🎵

Systematic search with pruning — how backtracking powers solutions to constraint satisfaction, configuration, and combinatorial problems.

Some problems just can't be solved with a clean formula. You need to try things, see if they work, and undo them when they don't. That's backtracking in a nutshell.

In AI coding interviews, backtracking shows up a lot through problems like configuration generators, scheduling problems, puzzle solvers, and anything where you're searching through a combinatorial space for valid arrangements. These problems are perfect for this format because the solution structure is complex enough to be interesting but follows a recognizable template that you and the AI can collaborate on effectively.

The easiest way to picture it is you're walking a maze, pick a direction, and go until you hit a wall, then back up to the last fork and try another path. That's the whole pattern. Most backtracking problems are just dressed-up versions of this same thing.

You don't need to have backtracking memorized before walking into the interview. Interviewers want to see that you can recognize when a problem calls for systematic search, articulate that to the AI, and verify that the generated solution actually prunes correctly.

Recognizing the pattern

Backtracking usually announces itself in the prompt. If the problem says "find all valid configurations" or "generate all arrangements satisfying these rules," it's almost certainly backtracking. Scheduling problems where resources have constraints, configuration generators where options depend on previous choices, puzzle solvers with rules that restrict placement, these all follow the pattern.

These problems tend to fall into a few categories. Once you've seen each shape once, the rest is mostly recognition:

- **Placement problems:** arrange items in a grid or structure so they don't conflict. N-Queens, Sudoku, crossword puzzle generators.

- **Scheduling and assignment:** assign tasks to resources, time slots to events, or workers to shifts, subject to constraints like availability and capacity.
- **Configuration generation:** produce all valid configurations of a system. Think feature flag combinations that satisfy dependency rules, or network topologies that meet connectivity requirements.
- **Path and partition problems:** find paths through a maze, partition sets into groups meeting certain criteria, or generate all valid orderings of tasks with dependencies.

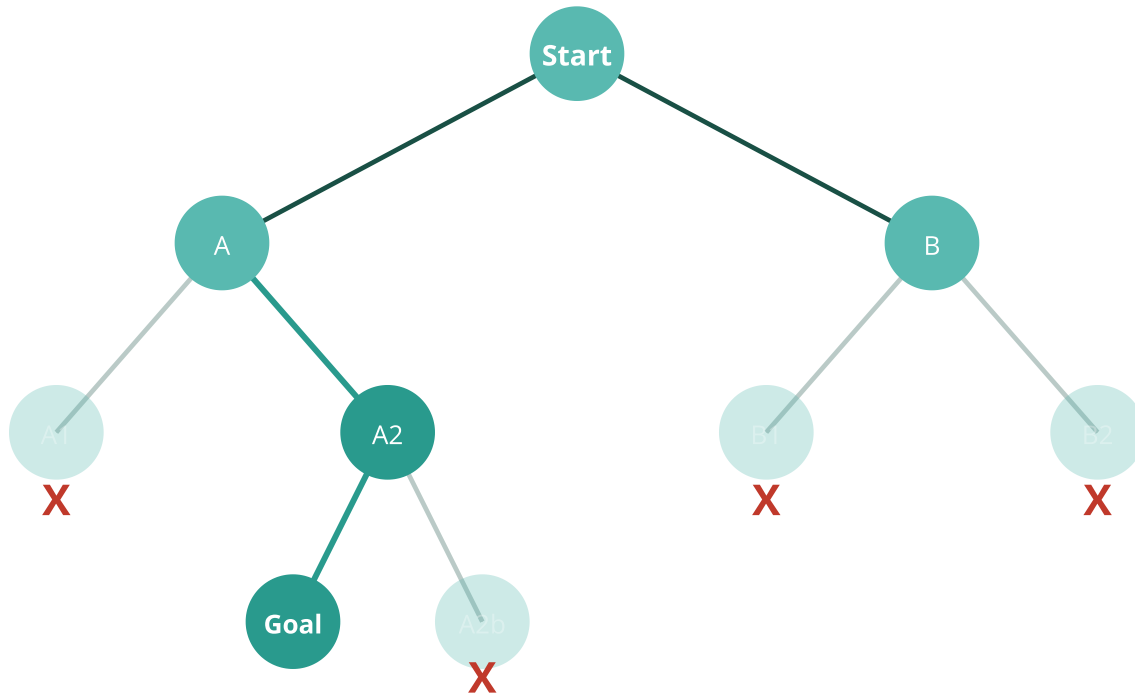
The common thread is you're building a solution incrementally, and at each step you can check whether you've already violated a constraint. If you have, there's no point continuing down that path.

Search with early termination

Backtracking is just trying every possible path until you find one that works, and abandoning paths the moment you know they won't.

Picture a tree where each branch is a choice you could make. You walk down one branch, making decisions as you go. If you hit a dead end (a choice that breaks a rule, or a path that can't lead to a solution), you back up to the last decision point and try a different branch. Then you keep going.

The maze version is the same idea. You pick a direction, walk until you hit a wall, then back up to the last fork and try another path. You don't restart from the beginning each time. You don't explore every inch of every dead end. You back up to where you still had options.



The highlighted path is the one the algorithm actually follows to a solution. The faded branches with red X marks are paths that got pruned because the algorithm checked a constraint, found a violation, and stopped exploring. Without pruning, you'd visit every single node. With it, you skip entire subtrees.

This is what makes backtracking practical. The theoretical search space for many of these problems is enormous, but good constraint checking cuts it down to something manageable.

The backtracking template

Almost every backtracking solution follows the same three-step rhythm: choose, explore, unchoose. You make a decision, recurse into it, and then undo that decision before trying the next option.

 Backtracking Template

Python  

```
def backtrack(state, choices):  
    if is_solution(state):  
        result.append(state.copy())  
    return
```

```
for choice in choices:
    if is_valid(state, choice):
        state.apply(choice)
        backtrack(state, choices)
        state.undo(choice)
```

That's it. Every backtracking problem is a variation on this skeleton. The differences are in what "state" looks like, how you define "choices" at each step, and what "is_valid" checks.

The `state.undo(choice)` line is the part humans often forget (AI almost never does), and it's the part that makes backtracking work. Without it, the decisions you made for one branch leak into the next branch, and everything breaks. When you're reviewing AI-generated backtracking code, it's useful to give a nod to this line in your narration to the interviewer. Them seeing you check for this critical piece is a strong signal you know what's going on.

Worked example: N-Queens

The N-Queens problem is the classic backtracking example for good reason. Place N queens on an N x N chessboard so that no two queens threaten each other. A queen attacks any square in the same row, the same column, or along either diagonal (a queen can move any number of squares forward, backward, sideways, or diagonally), so a valid arrangement has no two queens sharing any of those.

For a 4x4 board, here's what one valid solution looks like:

	0	1	2	3
0		Q		
1				Q
2	Q			
3			Q	

Queens are at positions (0,1), (1,3), (2,0), and (3,2). No two share a row, column, or diagonal.

Why is this a good backtracking problem? Because the constraints are tight. Each queen you place eliminates an entire row, column, and two diagonals. By the time you're placing the third or fourth queen, most of the board is already off-limits. This means the search tree gets pruned heavily, and the algorithm finishes quickly despite the theoretically huge number of possible arrangements (for an 8x8 board, there are over 4 billion ways to place 8 queens, but only 92 valid solutions).

Go row by row. For each row, scan the columns left to right; the first one that doesn't conflict with queens already on the board is where this row's queen goes, then you recurse into the next row. If you exhaust the columns without finding a safe one, undo the queen on the previous row and shift it to its next option.

See if you can read this code quickly (30 seconds) and notice the choose-explore-unchoose pattern. That's what you'll be doing in the interview!

N-Queens Solver

Python  

```
def solve_n_queens(n: int) -> list[list[int]]:
    solutions = []
    columns = set()
    diag1 = set()
    diag2 = set()

    def backtrack(row: int, placement: list[int]):
        if row == n:
            solutions.append(placement[:])
            return

        for col in range(n):
            if col in columns or (row - col) in diag1 or (row + col) in diag2:
                continue

            columns.add(col)
            diag1.add(row - col)
            diag2.add(row + col)
            placement.append(col)

            backtrack(row + 1, placement)

            columns.remove(col)
            diag1.remove(row - col)
            diag2.remove(row + col)
            placement.pop()

        backtrack(0, [])
    return solutions
```

For each column we try, we add it to our tracking sets (choose), recurse (explore), then remove it (unchoose). The three sets (`columns` , `diag1` , and `diag2`) let us check constraints in $O(1)$ instead of scanning the whole board each time.

The diagonal trick is that two cells share a diagonal if their `row - col` values match (top-left to bottom-right diagonals) or their `row + col` values match (top-right to bottom-left diagonals). This is the kind of thing that's fine to let the AI generate, but you should be able to explain it if the interviewer asks.



You don't need to have the diagonal formula memorized going in. If you tell the AI "use sets to track occupied columns and diagonals for O(1) conflict checking," it'll produce something like this. The interviewer cares that you knew to ask for efficient constraint checking, not that you remembered the exact math.

Constraint propagation: making pruning aggressive

Basic backtracking checks constraints when you make a choice. Constraint propagation goes further: after making a choice, it immediately figures out what that choice implies for future choices and eliminates options that are no longer viable.

Think about Sudoku. When you place a 5 in a cell, basic backtracking just checks "is this valid right now?" Constraint propagation also removes 5 from the possible values for every cell in the same row, column, and 3x3 box. When a cell gets reduced to a single possible value, that value gets placed automatically, which triggers more propagation.

When constraints are tight, meaning each choice eliminates a lot of future options, propagation makes backtracking dramatically faster. Problems that seem exponential on paper become nearly linear in practice because the search tree gets pruned so aggressively.

When constraints are loose, though, propagation doesn't help much. If each choice barely affects future options, you're back to brute-force search through an enormous tree. This is when you need to think about whether backtracking is even the right approach, or if you'd be better off with dynamic programming or a greedy strategy.

Try to quickly read through this code and notice the constraint propagation.



Sudoku Solver with Constraint Propagation

Python



```
def solve_sudoku(board: list[list[int]]) -> bool:
    possible = [[set(range(1, 10)) for _ in range(9)] for _ in range(9)]
```

```

    for r in range(9):
        for c in range(9):
            if board[r][c] != 0:
                propagate(possible, r, c, board[r][c])

    return backtrack_sudoku(board, possible)

def propagate(possible, row, col, val):
    for i in range(9):
        possible[row][i].discard(val)
        possible[i][col].discard(val)
    box_r, box_c = 3 * (row // 3), 3 * (col // 3)
    for r in range(box_r, box_r + 3):
        for c in range(box_c, box_c + 3):
            possible[r][c].discard(val)

def backtrack_sudoku(board, possible) -> bool:
    cell = find_empty_cell(board)
    if cell is None:
        return True

    row, col = cell
    for val in list(possible[row][col]):
        if is_valid_placement(board, row, col, val):
            board[row][col] = val
            saved = [[cell.copy() for cell in r] for r in possible]
            propagate(possible, row, col, val)

            if backtrack_sudoku(board, possible):
                return True

            board[row][col] = 0
            for i in range(9):
                for j in range(9):
                    possible[i][j] = saved[i][j]

    return False

```

One key detail is when we backtrack, we restore the entire `possible` state. This is the same choose-explore-unchoose pattern, just with more state to manage. The more state, the more likely an AI is to make a mistake here. So that's your number one thing to verify.



When reviewing AI-generated constraint propagation code, always check state restoration. The AI might correctly propagate constraints forward but fail to undo them on backtrack. This produces solutions that seem to work on small inputs but silently break on larger ones because stale constraints from abandoned branches contaminate the search.

Prompting the AI

The most useful thing you can do before prompting is describe the decision tree out loud. What choices are you making at each step, what constraints are you checking, when are you pruning? "I'll try placing items one at a time, checking constraints after each placement, and backtracking when I hit a violation" tells the interviewer you have a mental model — and the same sentence, lightly rewritten, is the prompt that gets the AI to generate clean code on the first try.

Even when your actual prompt is short ("write a backtracking solution for N-Queens"), narrate the underlying logic as you go. "I'm going to write a backtracking solution that places queens row by row, tracking occupied columns and diagonals in sets, and skips any column that conflicts" helps in two directions: the output is better, and you spend less time fixing it.

One pattern-specific move worth knowing: when the constraint-checking is non-trivial, name the data structure you want for `O(1)` validity checks. "Track occupied columns and diagonals in three sets" produces sharper code than "check whether the placement is valid." The AI will pick something reasonable either way, but the explicit version is faster and easier to verify.

Verifying the AI's code

After the AI generates the code, verify three things:

1. **State restoration** — does every modification to shared state get undone after the recursive call? Look for the unchoose step. If the code modifies a set, list, or array before recursing, that same modification should be reversed after recursing.
2. **Correct pruning** — does the `is_valid` check actually catch all constraint violations? Missing a constraint means the algorithm explores branches it shouldn't, which usually produces wrong answers rather than just being slow.

3. **Termination** — does every recursive path eventually hit a base case? With backtracking, infinite loops typically come from the recursion not actually making progress. Each recursive call should be moving toward a smaller problem (fewer remaining choices, fewer empty cells, one more item placed).



A good way to test backtracking code is with small inputs where you can manually verify the answer. For N-Queens, $n=4$ has exactly 2 solutions. For Sudoku, use a board with only a few empty cells. If the algorithm produces the right count or the right answer on a small instance, it's probably correct. If it hangs or gives the wrong count, you know to look at pruning or state restoration.

Asking the AI to generate some small inputs to validate correctness is a good idea with a high "bang for the buck".

When to use vs. alternatives

Backtracking is the right tool when you need to search a space of combinations and constraints let you prune aggressively. But it's not always the best choice.

Use backtracking when:

- You need all valid solutions, not just one
- Constraints are tight enough that most branches get pruned early
- The problem involves placing, assigning, or arranging items subject to rules
- There's no overlapping substructure (which would point toward DP)

Consider dynamic programming instead when:

- The problem has optimal substructure and overlapping subproblems
- You're optimizing a single value (minimum cost, maximum profit) rather than enumerating solutions
- You find yourself solving the same subproblem repeatedly during backtracking

Consider greedy instead when:

- A locally optimal choice at each step leads to a globally optimal solution

- You don't need to explore alternatives; the first valid choice is always the best

The tricky cases are problems that look like they need backtracking but actually have DP solutions. Subset sum, for instance, can be solved either way. Backtracking explores all subsets that might sum to the target. DP builds up from smaller sums. The DP approach is almost always faster, but it requires recognizing the overlapping substructure, which isn't always obvious.

There's also a middle ground worth knowing about: backtracking with memoization. If your backtracking solution visits the same state through different paths, you can cache results for states you've already fully explored. This is essentially how top-down DP works. If you notice your backtracking solution is slow and you suspect repeated work, mention this to the interviewer.



If you're unsure whether a problem needs backtracking or DP, start by asking: "Am I looking for all valid solutions, or just the optimal one?" Enumeration problems (all permutations, all valid configurations) usually want backtracking. Optimization problems (shortest path, minimum cost) usually want DP. There are exceptions, but this heuristic gets you started in the right direction.

In an AI coding interview, it's completely fine to say "I think this is a backtracking problem because we need to enumerate valid configurations, but I want to double-check that there isn't a DP formulation that would be more efficient." That kind of reasoning out loud is exactly what interviewers want to hear. It shows you're thinking about algorithmic tradeoffs rather than pattern-matching to the first thing that comes to mind.

What interviewers expect

A few specific signals separate a strong candidate from a weak one on backtracking problems:

- **You name the pattern out loud, in the right vocabulary.** "This needs systematic search with pruning" or "this is a backtracking problem" — said early, before you touch the keyboard — tells the interviewer you've recognized the shape.
- **You describe the decision tree before the AI writes code.** What's the choice at each step, what state are you tracking, what's the constraint that lets you prune? If you can't say

it, you can't verify the AI's output against it.

- **You catch the bugs that matter.** Walking through state restoration, pruning correctness, or termination out loud after the AI generates code is the difference between reading what the model produced and understanding what it produced.
- **You can articulate why backtracking, not something else.** If the interviewer pushes back with "would DP work here?" you should be able to answer concretely — "no, because we need to enumerate all solutions, not optimize a single value" — not just default to backtracking because the code looks recursive.

Interviewers using the AI-enabled format aren't expecting you to have every backtracking variant committed to memory. They're testing whether you can identify the pattern, communicate a plan, direct the AI effectively, catch the bugs that matter, and keep moving.

Putting it together

Backtracking problems in AI coding interviews follow a predictable arc. You recognize the combinatorial search structure, describe the decision tree and constraints to the interviewer, prompt the AI with enough specificity that it generates clean code, and then verify the critical details: state restoration, correct pruning, and termination.

The pattern itself is simple. Choose, explore, unchoose. The complexity comes from the specific constraints of each problem and how aggressively you can prune. The real problem is to solve something where part of the solution may involve a backtracking algorithm, so **keep moving**.

Test Your Knowledge

Take a quick 15 question quiz to test what you've learned and identify gaps.

 Start Quiz